Numerisches Lösen

Bisektionsverfahren

Name: Konstacky Julian

Klasse: 3BHWII

Schuljahr: 2025/26

Gegenstand: Softwareentwicklung und Projektmanagement (SWP)

Aufgabe 5 – Bisektions-SOLVER

Aufgabenstellung

Implementierung eines SOLVER für stetige Funktionen f mit einer Variablen x mittels der Methode der Intervallhalbierung (Bisektion). Getestet mit der Wurzelfunktion für n = 25, n = 81 und n = 144.

Beschreibung

Der SOLVER nutzt das Bisektionsverfahren, um die Nullstelle einer stetigen Funktion im Intervall [a, b] iterativ zu finden. Voraussetzung ist ein Vorzeichenwechsel im Intervall (Zwischenwertsatz): $f(a) \cdot f(b) < 0$. In jedem Schritt wird der Mittelpunkt $c = (a+b)/2$ berechnet und je nach Vorzeichen von f(c) wird das Intervall auf [a,c] oder [c,b] halbiert. Dies wird wiederholt bis $|f(c)| < \varepsilon$.

Für die Wurzelfunktion gilt: Gesucht ist x mit $x^2 = n \rightarrow$ Nullstellenproblem: $f(x) = x^2 - n = 0$

Datei: Aufgabe5.py | Ausführung: python Aufgabe5.py

Quellcode

```
"""
Aufgabe 5: Bisektions-SOLVER
Numerisches Loesen von Nullstellen-Problemen mittels Intervallhalbierung.
"""

import sys
import math
from typing import Callable, Tuple, List

sys.stdout.reconfigure(encoding="utf-8")



def bisect(
    func: Callable[[float], float],
    a: float,
    b: float,
    epsilon: float = 1e-10,
    max_iter: int = 1000
) -> Tuple[float, int, List[Tuple[float, float]]]:
    """
    Bisektionsverfahren zur Nullstellenfindung in [a, b].
    Voraussetzung: f(a) * f(b) < 0 (Zwischenwertsatz).
    """
    try:
        fa: float = func(a)
        fb: float = func(b)
    except Exception as e:
        raise ValueError(f"Fehler bei der Funktionsauswertung: {e}")

    if fa * fb >= 0:
        raise ValueError(
            f"Zwischenwertsatz nicht erfuellt: f({a})={fa:.6f} und "
            f"f({b})={fb:.6f} haben gleiches Vorzeichen."
        )

    history: List[Tuple[float, float]] = []
    c: float = a

    for i in range(max_iter):
        c = (a + b) / 2.0            # Mittelpunkt des Intervalls
```

```
        fc: float = func(c)
        history.append((c, abs(fc)))

        if abs(fc) < epsilon or (b - a) / 2.0 < epsilon:
            return c, i + 1, history

        if fa * fc < 0:
            b, fb = c, fc
        else:
            a, fa = c, fc

    raise RuntimeError(f"Maximale Iterationszahl ({max_iter}) erreicht.")


def sqrt_func(n: float) -> Callable[[float], float]:
    """Erstellt f(x) = x^2 - n fuer die Wurzelberechnung sqrt(n)."""
    return lambda x: x ** 2 - n


def solver() -> None:
    """Demonstriert Bisektions-SOLVER mit Wurzelfunktion fuer n=25, 81, 144."""
    print("=" * 65)
    print("Aufgabe 5: Bisektions-SOLVER  -  Wurzelfunktions-Test")
    print("=" * 65)

    test_cases: List[Tuple[float, float, float]] = [
        (25,  0.0,  25.0),
        (81,  0.0,  81.0),
        (144, 0.0, 144.0),
    ]

    for n, a, b in test_cases:
        try:
            root, iterations, _ = bisect(sqrt_func(n), a, b)
            analytical: float = math.sqrt(n)
            error: float = abs(root - analytical)
            print(f"\nsqrt({n:3.0f}):")
            print(f"  Numerisch  = {root:.10f}")
            print(f"  Analytisch = {analytical:.10f}")
            print(f"  Fehler     = {error:.2e}")
            print(f"  Iterationen= {iterations}")
        except (ValueError, RuntimeError) as e:
            print(f"\nFehler fuer n={n}: {e}")

    print("\n" + "=" * 65)


if __name__ == "__main__":
    solver()
```

Testergebnisse

```
=================================================================
Aufgabe 5: Bisektions-SOLVER  -  Wurzelfunktions-Test
=================================================================


sqrt( 25):
  Numerisch  = 5.0000000000
  Analytisch = 5.0000000000
  Fehler     = 1.82e-11
  Iterationen= 38
```

```
sqrt( 81):
  Numerisch  = 8.9999999999
  Analytisch = 9.0000000000
  Fehler     = 5.73e-11
  Iterationen= 40

sqrt(144):
  Numerisch  = 12.0000000000
  Analytisch = 12.0000000000
  Fehler     = 2.18e-11
  Iterationen= 41

================================================================
```

Interpretation

Die numerischen Ergebnisse stimmen mit den analytischen Lösungen ($\sqrt{25} = 5$, $\sqrt{81} = 9$, $\sqrt{144} = 12$) auf mindestens 10 Dezimalstellen überein. Der maximale Fehler beträgt $5{,}73 \times 10^{-11}$, was weit unter der gesetzten Genauigkeit von $\varepsilon = 10^{-10}$ liegt. Für sqrt(25) werden 38 Iterationen benötigt, für sqrt(81) 40 und für sqrt(144) 41 Iterationen.

Aufgabe 6 – Regula Falsi

Aufgabenstellung

Alternative Lösungsmethode zur Bisektion mittels Regula Falsi (Methode der falschen Position). Getestet mit denselben Werten wie Aufgabe 5.

Beschreibung

Beim Regula-Falsi-Verfahren wird statt des Mittelpunkts eine Sekante durch die Punkte (a, f(a)) und (b, f(b)) gelegt. Die Nullstelle der Sekante dient als neue Intervallgrenze:

$$c = b - f(b) \cdot (b - a) / (f(b) - f(a))$$

Ziel ist eine schnellere Konvergenz als bei der reinen Intervallhalbierung. Die Vorzeichenbedingung bleibt dieselbe wie bei der Bisektion.

Datei: Aufgabe6.py | Ausführung: python Aufgabe6.py

Quellcode

```
"""
Aufgabe 6: Regula Falsi (Methode der falschen Position)
Alternative zum Bisektionsverfahren aus Aufgabe 5.
"""

import sys
import math
from typing import Callable, Tuple, List
from Aufgabe5 import sqrt_func

sys.stdout.reconfigure(encoding="utf-8")



def regula_falsi(
    func: Callable[[float], float],
    a: float,
    b: float,
    epsilon: float = 1e-10,
    max_iter: int = 1000
) -> Tuple[float, int, List[Tuple[float, float]]]:
    """
    Regula-Falsi: c = b - f(b) * (b-a) / (f(b) - f(a))
    """
    try:
        fa: float = func(a)
        fb: float = func(b)
    except Exception as e:
        raise ValueError(f"Fehler bei der Funktionsauswertung: {e}")

    if fa * fb >= 0:
        raise ValueError(
            f"Zwischenwertsatz nicht erfuellt: f({a})={fa:.6f}, f({b})={fb:.6f}."
        )

    history: List[Tuple[float, float]] = []
    c: float = b

    for i in range(max_iter):
        denom: float = fb - fa
        if abs(denom) < 1e-15:
```

```
            raise ValueError("Division durch Null: f(b)-f(a) nahezu 0.")

        c = b - fb * (b - a) / denom   # Sekantenschnittpunkt
        fc: float = func(c)
        history.append((c, abs(fc)))

        if abs(fc) < epsilon:
            return c, i + 1, history

        if fa * fc < 0:
            b, fb = c, fc
        else:
            a, fa = c, fc

    raise RuntimeError(f"Maximale Iterationszahl ({max_iter}) erreicht.")


def solver2() -> None:
    """Demonstriert Regula Falsi mit Wurzelfunktion fuer n=25, 81, 144."""
    print("=" * 65)
    print("Aufgabe 6: Regula Falsi  -  Wurzelfunktions-Test")
    print("=" * 65)

    for n, a, b in [(25, 0.0, 25.0), (81, 0.0, 81.0), (144, 0.0, 144.0)]:
        try:
            root, iterations, _ = regula_falsi(sqrt_func(n), a, b)
            analytical = math.sqrt(n)
            print(f"\nsqrt({n:3.0f}):")
            print(f"  Numerisch  = {root:.10f}")
            print(f"  Analytisch = {analytical:.10f}")
            print(f"  Fehler     = {abs(root - analytical):.2e}")
            print(f"  Iterationen= {iterations}")
        except (ValueError, RuntimeError) as e:
            print(f"\nFehler: {e}")

    print("\n" + "=" * 65)


if __name__ == "__main__":
    solver2()
```

Testergebnisse

```
=================================================================
Aufgabe 6: Regula Falsi  -  Wurzelfunktions-Test
=================================================================

sqrt( 25):
  Numerisch  = 5.0000000000
  Analytisch = 5.0000000000
  Fehler     = 7.08e-12
  Iterationen= 69

sqrt( 81):
  Numerisch  = 9.0000000000
  Analytisch = 9.0000000000
  Fehler     = 4.53e-12
  Iterationen= 130

sqrt(144):
  Numerisch  = 12.0000000000
```

```
  Analytisch = 12.0000000000
  Fehler     = 4.09e-12
  Iterationen= 176

==================================================================
```

Interpretation

Das Regula-Falsi-Verfahren liefert eine höhere Genauigkeit (Fehler $\sim 4 \times 10^{-12}$ gegenüber $\sim 6 \times 10^{-11}$ bei der Bisektion), benötigt dafür aber deutlich mehr Iterationen (69–176 gegenüber 38–41). Dies liegt daran, dass die Sekante bei der Wurzelfunktion einseitig konvergiert und eine Intervallgrenze eingefroren bleibt (langsame Konvergenz des Regula Falsi).

Aufgabe 7 – Grafische Visualisierung mit Matplotlib

Aufgabenstellung

Animierte grafische Aufbereitung der Nullstellenfindung mittels Matplotlib. Zwei Subplots in einer Figure: Aktuelle Genauigkeit und aktuelle Lösung je Iterationsschritt.

Beschreibung

Die Funktion animate_solver() erstellt eine Figure mit zwei Subplots und aktualisiert diese nach jedem Iterationsschritt:

- Subplot 1 (links): |f(c)| je Iteration auf logarithmischer Skala – nähert sich dem Wert 0.
- Subplot 2 (rechts): Aktueller Näherungswert c je Iteration – nähert sich der analytischen Lösung.

plt.close('all') stellt sicher, dass kein altes Fenster offen ist. plt.pause() aktualisiert die Figure in-place ohne neue Fenster zu öffnen. plt.show(block=True) hält das Fenster offen bis der Nutzer es schließt. Die Funktion plotter() demonstriert beide Verfahren nacheinander für sqrt(25).

Datei: Aufgabe7.py | Ausführung: python Aufgabe7.py

Quellcode

```
"""
Aufgabe 7: Grafische Aufbereitung der Nullstellenfindung
Animierte Visualisierung mit Matplotlib (zwei Subplots).
"""

import sys
import matplotlib.pyplot as plt
import numpy as np
from typing import List, Tuple
from Aufgabe5 import bisect, sqrt_func
from Aufgabe6 import regula_falsi

sys.stdout.reconfigure(encoding="utf-8")



def animate_solver(
    history: List[Tuple[float, float]],
    true_root: float,
    title: str,
    pause: float = 0.08
) -> None:
    """
    Animiert Nullstellenfindung: 2 Subplots, ein Fenster, kein Flimmern.
    Subplot 1: |f(c)| je Iteration (log. Skala).
    Subplot 2: Naehrung c je Iteration.
    """
    steps = list(range(1, len(history) + 1))
    c_values      = [h[0] for h in history]
    accuracy_vals = [h[1] for h in history]

    plt.close("all")   # Kein altes Fenster offen lassen
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))
    fig.suptitle(title, fontsize=13, fontweight="bold")

    for i in range(len(history)):
```

```
        ax1.cla()
        ax1.semilogy(steps[:i+1], accuracy_vals[:i+1], "b-o", markersize=4)
        ax1.set_xlabel("Iterationsschritt")
        ax1.set_ylabel("|f(c)|  (log. Skala)")
        ax1.set_title("Aktuelle Genauigkeit je Iterationsschritt")
        ax1.set_xlim(1, max(len(history), 2))
        ax1.grid(True, which="both", linestyle="--", alpha=0.5)

        ax2.cla()
        ax2.plot(steps[:i+1], c_values[:i+1], "r-o", markersize=4, label="Naehrung c")
        ax2.axhline(y=true_root, color="green", linestyle="--",
                    label=f"Loesung = {true_root:.4f}")
        ax2.set_xlabel("Iterationsschritt")
        ax2.set_ylabel("c")
        ax2.set_title("Aktuelle Naehrung c je Iterationsschritt")
        ax2.set_xlim(1, max(len(history), 2))
        ax2.legend(loc="upper right")
        ax2.grid(True, linestyle="--", alpha=0.5)

        plt.tight_layout()
        plt.pause(pause)   # In-place aktualisieren - kein neues Fenster

    plt.show(block=True)   # Offen halten bis Nutzer schliesst


def plotter() -> None:
    """Demonstriert Animation fuer Bisektion und Regula Falsi mit sqrt(25)."""
    n, a, b = 25.0, 0.0, 25.0
    func      = sqrt_func(n)
    true_root = float(np.sqrt(n))

    _, _, hist_b = bisect(func, a, b, epsilon=1e-8)
    print(f"Bisektion fuer sqrt({n:.0f}): {len(hist_b)} Iterationen")
    animate_solver(hist_b, true_root,
        title=f"Bisektion - sqrt({n:.0f})  (Nullstelle bei x = {true_root:.4f})")

    _, _, hist_rf = regula_falsi(func, a, b, epsilon=1e-8)
    print(f"Regula Falsi fuer sqrt({n:.0f}): {len(hist_rf)} Iterationen")
    animate_solver(hist_rf, true_root,
        title=f"Regula Falsi - sqrt({n:.0f})  (Nullstelle bei x = {true_root:.4f})")


if __name__ == "__main__":
    plotter()
```

Die Visualisierung zeigt für jeden Iterationsschritt die aktuelle Näherung und die Genauigkeit. Beide Fenster werden nacheinander angezeigt – erst Bisektion, dann Regula Falsi.

Aufgabe 8 – Polynom P4(x)

Aufgabenstellung

Nullstellenfindung des Polynoms $P4(x) = 2x + x^2 + 3x^3 - x^4$ bei $x \approx 3{,}4567$ mit dem SOLVER aus Aufgabe 5. Genauigkeitstests für $\varepsilon = 10^{-2}$ und $\varepsilon = 10^{-8}$.

Intervallwahl

$P4(3) = 2 \cdot 3 + 9 + 81 - 81 = 15 > 0$

$P4(4) = 2 \cdot 4 + 16 + 192 - 256 = -40 < 0$

Vorzeichenwechsel in [3, 4] → Nullstelle garantiert (Zwischenwertsatz). Gewähltes Intervall: [a, b] = [3, 4]

Datei: Aufgabe8.py | Ausführung: python Aufgabe8.py

Quellcode

```
"""
Aufgabe 8: Polynom P4(x) = 2x + x^2 + 3x^3 - x^4
Nullstellenfindung und Genauigkeitstests mit dem Bisektions-SOLVER.
"""

import sys
from Aufgabe5 import bisect

sys.stdout.reconfigure(encoding="utf-8")



def p4(x: float) -> float:
    """Polynom P4(x) = 2x + x^2 + 3x^3 - x^4."""
    return 2 * x + x ** 2 + 3 * x ** 3 - x ** 4



def run_aufgabe8() -> None:
    """Nullstelle von P4 bei x~3.4567, Genauigkeitstests eps=10^-2 und 10^-8."""
    print("=" * 65)
    print("Aufgabe 8: Polynom  P4(x) = 2x + x^2 + 3x^3 - x^4")
    print("=" * 65)

    a, b = 3.0, 4.0   # P4(3)=15 > 0, P4(4)=-40 < 0 --> Vorzeichenwechsel
    print(f"\nPruefe Intervall [{a}, {b}]:")
    print(f"  P4({a}) = {p4(a):.6f}")
    print(f"  P4({b}) = {p4(b):.6f}")

    try:
        root, iters, _ = bisect(p4, a, b)
        print(f"\nNullstelle: x ~ {root:.6f}  (erwartet: x ~ 3.4567)")
        print(f"P4(x)      = {p4(root):.2e}")
        print(f"Iterationen: {iters}")

        print("\nGenauigkeitstests:")
        print(f"  {'epsilon':<12} {'Iterationen':>12} {'x (Naehrung)':>18}
{'|f(x)|':>12}")
        print("  " + "-" * 56)

        for exp in [-2, -8]:
            eps = 10.0 ** exp
```

```
            r, it, _ = bisect(p4, a, b, epsilon=eps)
            print(f"  10^{exp:<8} {it:>12}    {r:>16.10f}   {abs(p4(r)):>12.2e}")

    except (ValueError, RuntimeError) as e:
        print(f"Fehler: {e}")

    print("\n" + "=" * 65)
    print("Interpretation:")
    print("  Fuer eps=10^-2 genuegen 7 Iterationen.")
    print("  Fuer eps=10^-8 werden 27 Iterationen benoetigt.")
    print("  Pro Iteration halbiert sich das Intervall --> log2-Konvergenz.")


if __name__ == "__main__":
    run_aufgabe8()
```

Testergebnisse

```
=================================================================
Aufgabe 8: Polynom  P4(x) = 2x + x^2 + 3x^3 - x^4
=================================================================

Pruefe Intervall [3.0, 4.0]:
  P4(3.0) = 15.000000
  P4(4.0) = -40.000000

Nullstelle: x ~ 3.456678  (erwartet: x ~ 3.4567)
P4(x)      = -9.54e-10
Iterationen: 34

Genauigkeitstests:
  epsilon       Iterationen       x (Naehrung)       |f(x)|
  -------------------------------------------------------
  10^-2                  7        3.4609375000       2.08e-01
  10^-8                 27        3.4566783383       2.29e-07

=================================================================
Interpretation:
  Fuer eps=10^-2 genuegen 7 Iterationen.
  Fuer eps=10^-8 werden 27 Iterationen benoetigt.
  Pro Iteration halbiert sich das Intervall --> log2-Konvergenz.
```

Interpretation der Genauigkeitstests

Für $\varepsilon = 10^{-2}$: 7 Iterationen sind ausreichend, da die Genauigkeitsanforderung gering ist. Für $\varepsilon = 10^{-8}$: 27 Iterationen nötig. Das Verhältnis entspricht dem theoretischen Wert: $\log_2(10^2) \approx 6{,}6 \rightarrow 7$ Iterationen und $\log_2(10^8) \approx 26{,}6 \rightarrow 27$ Iterationen. Die Bisektion konvergiert logarithmisch zur Basis 2 – pro zusätzliches Bit Genauigkeit wird genau eine weitere Iteration benötigt.

Aufgabe 9 – Kettenlinie / Elektrische Leitung

Aufgabenstellung

Berechnung der Länge einer elektrischen Leitung zwischen zwei gleich hohen Masten, die 100 m auseinanderliegen, mit einem maximalen Durchhang von 10 m in der Mitte.

Mathematische Herleitung

Kettenlinie (mit $x_0 = 0$):

$$y(x) = a \cdot \cosh(x/a) - a + y_0$$

Randbedingung (Masten sind 10 m höher als der Scheitelpunkt):

$$y(50) = y_0 + 10$$

Einsetzen:

$$a \cdot \cosh(50/a) - a + y_0 = y_0 + 10$$

$$a \cdot \cosh(50/a) - a = 10$$

$$\rightarrow f(a) = a \cdot \cosh(50/a) - a - 10 = 0 \quad \text{(Nullstellenproblem für a!)}$$

Intervallwahl für den Krümmungsradius a:

$$f(50) = 50 \cdot \cosh(1) - 50 - 10 \approx +17{,}15 \quad \text{(positiv)}$$

$$f(1000) = 1000 \cdot \cosh(0{,}05) - 1000 - 10 \approx -8{,}75 \quad \text{(negativ)}$$

→ Vorzeichenwechsel in [50, 1000] → Nullstelle garantiert

Leitungslänge nach Findung von a:

$$l = 2a \cdot \sinh(w / 2a) = 2a \cdot \sinh(50/a)$$

Datei: Aufgabe9.py | Ausführung: python Aufgabe9.py

Quellcode

```
"""
Aufgabe 9: Kettenlinie - Laenge einer elektrischen Leitung
Masten: 100 m Abstand, 10 m Durchhang.
f(a) = a*cosh(50/a) - a - 10 = 0  --> Nullstellenproblem
"""

import sys
import math
from typing import Tuple, List
from Aufgabe5 import solver, bisect
from Aufgabe6 import solver2, regula_falsi
from Aufgabe7 import plotter

sys.stdout.reconfigure(encoding="utf-8")


def catenary_func(a: float) -> float:
    """
    Nullstellenfunktion fuer Kruemmungsradius a.
    Aus y(50) = y0 + 10 folgt: f(a) = a*cosh(50/a) - a - 10 = 0
```

```
    """
    return a * math.cosh(50.0 / a) - a - 10.0


def compute_cable_length(a: float) -> float:
    """Leitungslaenge: l = 2a * sinh(50/a)"""
    return 2.0 * a * math.sinh(50.0 / a)


def solve_catenary() -> None:
    """Berechnet Kruemmungsradius a und Leitungslaenge l."""
    print("=" * 65)
    print("Aufgabe 9: Kettenlinie - Leitungslaenge")
    print("=" * 65)
    print("Gegeben:  Mastabstand w = 100 m,  Durchhang = 10 m")
    print("Gesucht:  Kruemmungsradius a  und  Leitungslaenge l")
    print()
    print("Nullstellenproblem: f(a) = a*cosh(50/a) - a - 10 = 0")

    a_left, a_right = 50.0, 1000.0
    print(f"\nStartintervall [{a_left}, {a_right}]:")
    print(f"  f({a_left})    = {catenary_func(a_left):.4f}")
    print(f"  f({a_right}) = {catenary_func(a_right):.4f}\n")

    try:
        a_b, iters_b, _ = bisect(catenary_func, a_left, a_right)
        print("[Bisektion]")
        print(f"  Kruemmungsradius a = {a_b:.6f} m")
        print(f"  Leitungslaenge   l = {compute_cable_length(a_b):.6f} m")
        print(f"  Iterationen        = {iters_b}")
    except (ValueError, RuntimeError) as e:
        print(f"Fehler Bisektion: {e}")

    print()

    try:
        a_rf, iters_rf, _ = regula_falsi(catenary_func, a_left, a_right)
        print("[Regula Falsi]")
        print(f"  Kruemmungsradius a = {a_rf:.6f} m")
        print(f"  Leitungslaenge   l = {compute_cable_length(a_rf):.6f} m")
        print(f"  Iterationen        = {iters_rf}")
    except (ValueError, RuntimeError) as e:
        print(f"Fehler Regula Falsi: {e}")

    print("\n" + "=" * 65)


if __name__ == "__main__":
    solver()
    solver2()
    plotter()
    solve_catenary()
```

Testergebnisse

```
=================================================================
Aufgabe 9: Kettenlinie - Leitungslaenge
=================================================================
Gegeben:  Mastabstand w = 100 m,  Durchhang = 10 m
Gesucht:  Kruemmungsradius a  und  Leitungslaenge l
```

```
Nullstellenproblem: f(a) = a*cosh(50/a) - a - 10 = 0

Startintervall [50.0, 1000.0]:
  f(50.0)    = 17.1540
  f(1000.0) = -8.7497

[Bisektion]
  Kruemmungsradius a = 126.632436 m
  Leitungslaenge   l = 102.618687 m
  Iterationen        = 37

[Regula Falsi]
  Kruemmungsradius a = 126.632436 m
  Leitungslaenge   l = 102.618687 m
  Iterationen        = 61

================================================================
```

Interpretation

Beide Verfahren liefern denselben Krümmungsradius a ≈ 126,63 m. Die elektrische Leitung hat eine Länge von ca. l ≈ 102,62 m – also etwa 2,62 m mehr als der direkte Abstand von 100 m. Dies entspricht der physikalisch erwarteten Kettenlinie mit geringem Durchhang. Die Bisektion benötigt hier weniger Iterationen (37) als Regula Falsi (61), da die Funktion catenary_func stark nichtlinear ist und die Sekante des Regula Falsi schlechter konvergiert.